

Modellkomplexität und Modellauswahl

Contents

- Hyperparameter
- Modell-Komplexität
- Bias und Varianz (Variance)
- Modellauswahl
- Aufgaben

Lernziele

- Hyperparameter und ihren Effekt auf Modell-Performanz (am Beispiel von k -NN) vorstellen
- Zusammenhang zwischen Over-/Underfitting und Modell-Komplexität herstellen
- Fehler unterscheiden: Bias vs. Varianz
- Validierungsdaten und Cross-Validation für die Modellauswahl einsetzen

Hyperparameter

- Modelle, die wir trainieren, sollen i.d.R. **auf ungesehenen Datensätzen** gute Vorhersagen treffen (z.B. wir wollen nicht bereits bestätigte Diagnosen in gesehenen Patient*innen wiederentdecken sondern noch nicht bekannte

Erkrankungen rechtzeitig diagnostizieren können)

- Wir wollen sicherstellen, dass das trainierte Modell ein **relevantes Muster** in den Daten erkannt hat, und sich nicht lediglich möglichst viele Zusammenhänge in den Trainingsdaten "gemerkt" hat (mit genügend Freiheitsgraden ist dies machbar)
- Muster zu erkennen, die über die gesammelten Datenbeispiele hinweg **generalisieren**, ist ein fundamentales Problem im Machine Learning und der Statistik allgemein
- Da wir den wahren Fehler eines Modells nicht berechnen können, **schätzen wir ihn anhand der Testdaten**
- Bei vielen Lernverfahren gibt es Parameter, die vor dem eigentlichen training festgelegt werden müssen – bei k -NN das k . Solche Variablen heißen **Hyperparameter**
- Unser erstes Ziel ist zu verstehen, wie wir den Einfluss von Hyperparametern messen können und wie wir die optimalen Hyperparameter-Werte finden

Hyperparameter am Beispiel von k -NN

- Wie hängt die Performance eines k -NN Modells von k ab?
- Wir betrachten als Beispiel den Palmer Penguins Datensatz

```
import warnings
warnings.filterwarnings('ignore')

import pandas as pd
from os.path import join
import sys, os
import matplotlib.pyplot as plt

sys.path.append(os.path.abspath("src"))
from ds_helpers.plotting import setup_plots, wrap_labels
setup_plots()

from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
```

```
file_path = join('data', 'penguins_lter.csv')
penguins = pd.read_csv(file_path)

X_y = penguins[["Culmen Length (mm)", "Culmen Depth (mm)", "Body Mass (g)", "Species"]]
X_y.dropna(inplace=True)

species_map = {item:index for index,item in enumerate(X_y["Species"].unique())}
X_y["Species"] = X_y["Species"].map(species_map)

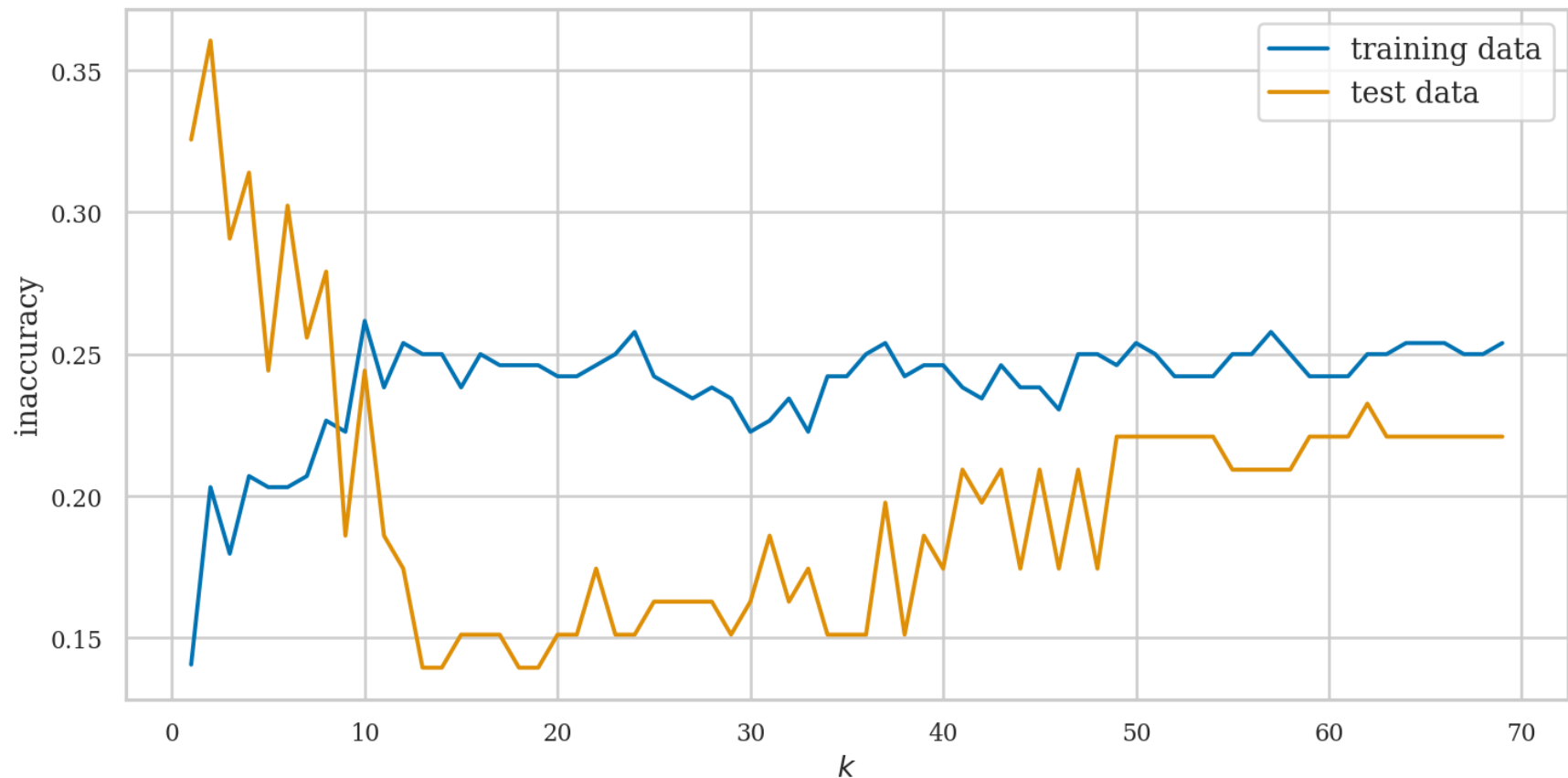
# wir ignorieren zunächst das Gewicht
X = X_y[["Culmen Length (mm)"]]
y = X_y["Species"]
```

```
def compute_train_test_error(X, y, n_neighbors):
    """Computes the training and the test error of a k-NN model on a standard train-test-split"""
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, stratify=y, random_state=66)
    clf = KNeighborsClassifier(n_neighbors=n_neighbors)
    clf.fit(X_train, y_train)
    return 1 - clf.score(X_train, y_train), 1 - clf.score(X_test, y_test)
```

```

fig, ax = plt.subplots(1, 1, figsize=(12,6))
train_errors = []
test_errors = []
neighbors_settings = range(1, 70)
for n_neighbors in neighbors_settings:
    train_error, test_error = compute_train_test_error(X, y, n_neighbors)
    train_errors.append(train_error)
    test_errors.append(test_error)
plt.plot(neighbors_settings, train_errors, label="training data" #, linestyle=':', marker='o'
plt.plot(neighbors_settings, test_errors, label="test data")
plt.ylabel("inaccuracy", fontsize=16)
plt.xlabel("$k$", fontsize=16)
plt.legend(fontsize=16);

```



- k klein: geringer Fehler auf den Trainingsdaten, hoher Fehler auf den Testdaten
- k groß: Testfehler (und Trainingsfehler) wächst bzw. stagniert
- Für $k \sim 13$: **niedrigster Fehler auf Testdaten**
- Wie können wir diesen Effekt erklären?
- Dafür betrachten wir die sog. **Entscheidungsgrenze** (engl: **decision boundary**)
- Kurve, welche den Feature-Raum X in zwei Mengen aufteilt, die jeweils den zwei Klassen entsprechen. D.h., das Modell ordnet alle Punkte in der einen Menge der einen Klasse zu und analog für die andere Menge
- Wenn wir mehr als 2 Klassen haben, teilt die Entscheidungsgrenze den Raum der Punkte in entsprechend mehr Teilmengen auf
- Wenn wir mehr als 2 Features haben (und entsprechend $\dim(X) > 2$), ist die Entscheidungsgrenze keine Kurve sondern eine sog. Hyperebene

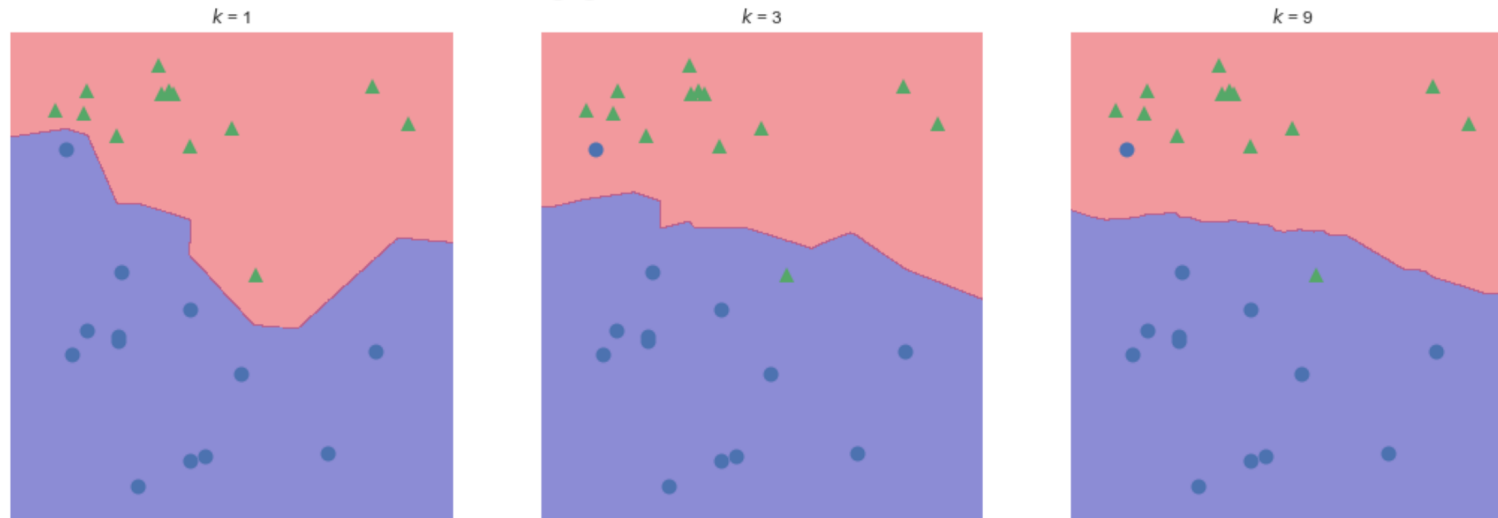
Wie sieht die Entscheidungsgrenze bei 1-NN aus?



Für $k = 1$:

- Jeder Punkt in den Trainingsdaten definiert seine eigene Einflussregion
- Die Entscheidungsgrenze schlängelt sich eng um alle Trainingsbeispiele herum – sie ist sehr **unregelmäßig** und **empfindlich gegenüber Rauschen**
- Das Modell ist **extrem flexibel**

- Jedes Trainingsbeispiel ist effektiv ein "Modell-Parameter" [\[1\]](#), der das lokale Klassifikationsverhalten bestimmt



Beispiel für Entscheidungsgrenzen von k -NN für verschiedene Werte von k .

größere Werte von k :

- Jede Vorhersage hängt von mehreren Trainingsbeispielen ab; lokale Schwankungen werden ausgeglichen
- Die Entscheidungsgrenze wird glatter; Vorhersage ist weniger empfindlich gegenüber einzelnen Trainingsbeispielen
- Die effektive Anzahl an Parametern sinkt; das Modell ist **weniger flexibel**

In anderen Worten: die **capacity** des Modells, beliebige Muster zu lernen fällt mit wachsendem k .

Diskussion (3–5 Min.)

Was passiert, wenn k gleich der Anzahl an Punkten in der Trainingsmenge ist?

► **Maximal viele Nachbarn**

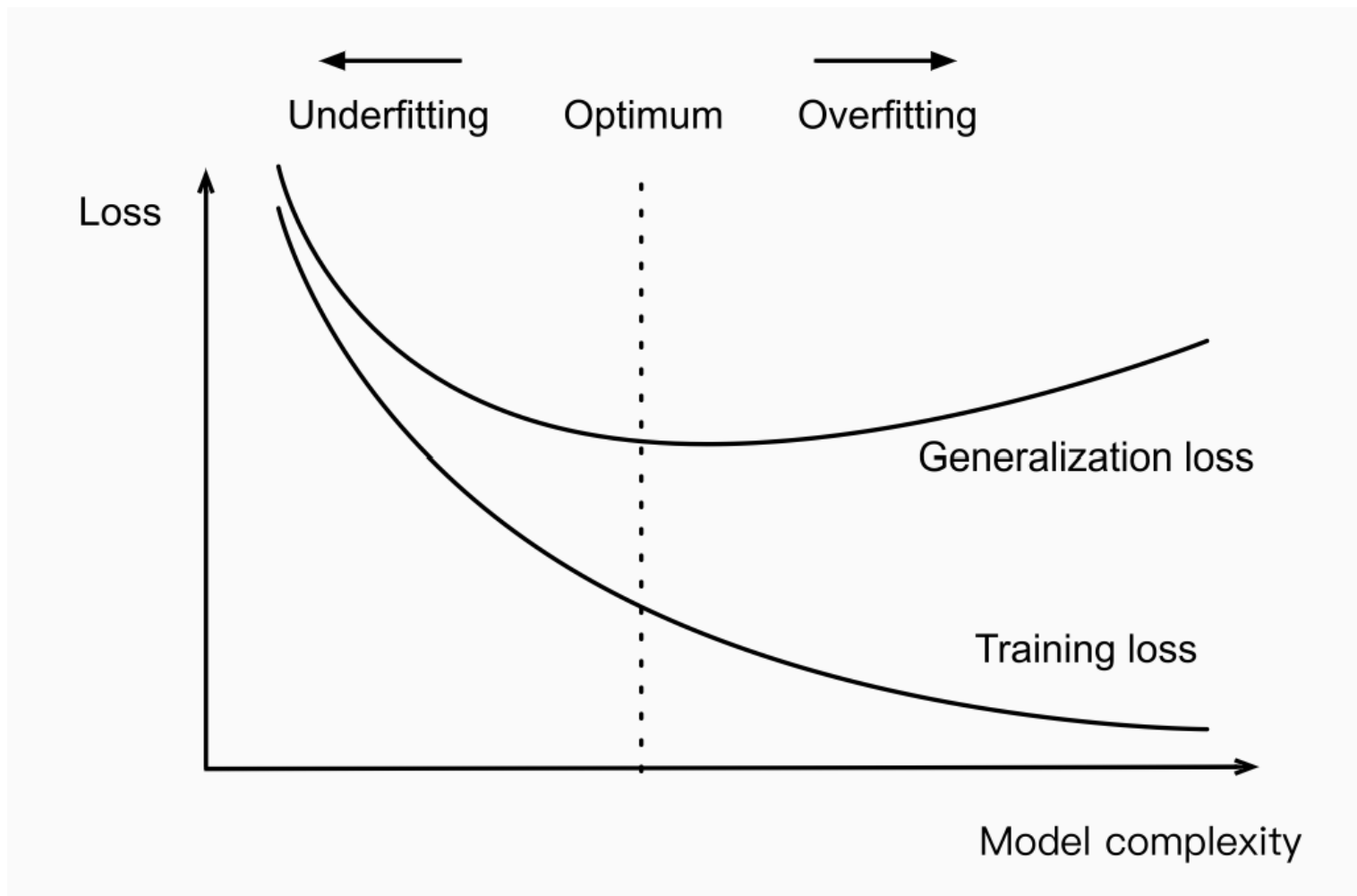
Jeder neue Punkt hätte die gleichen Nachbarn, und zwar alle Punkte in der Trainingsmenge, und die Vorhersage wäre für alle gleich, nämlich die am häufigsten vorkommende Klasse in den Trainingsdaten.

Modell-Komplexität

- Damit haben wir die zentrale Idee hinter der sog. **model complexity** kennengelernt: diese beschreibt, wie flexibel ein Modell ist, wie stark es sich also an Details und die Variabilität in den Daten anpassen kann
- Bei k -NN:
 - kleines $k \rightarrow$ sehr flexibel $\rightarrow$ höhere Komplexität
 - größeres $k \rightarrow$ weniger flexibel $\rightarrow$ glattere Entscheidungsgrenze, weniger komplexes Modell

Verhältnis zu Under- und Overfitting

- Underfitting (Unteranpassung): Modell zu einfach $\rightarrow$ Muster werden übersehen $\rightarrow$ hohe Fehlerquote bei Trainings- und Testdaten
- Overfitting (Überanpassung): Modell zu komplex $\rightarrow$ passt sich an Rauschen in den Trainingsdaten an $\rightarrow$ geringer Trainingsfehler, aber hoher Testfehler



Underfitting und Overfitting vs. Modell-Komplexität.

- **Modell-Komplexität** lässt sich nicht einfach definieren
- Vergleich unterschiedlicher Lernverfahren (Daumenregel): je mehr Parameter gelernt werden können, desto höher die Flexibilität der Modelle und somit die Anpassbarkeit an Trainingsdaten. Z.B.: Ein Polynom vom Grad 10 (viele Gewichte) kann sich durch jeden Datenpunkt schlängeln, während eine Gerade (=Polynom 1. Grades, wenige

Parameter) nur allgemeine Trends abbilden kann.

Bias und Varianz (Variance)

- Unter- bzw. Überangepasste Modelle machen unterschiedliche Arten von Fehlern - wir wollen beide minimieren
- Underfitting → hoher **Bias**, Overfitting → hohe **Variance**

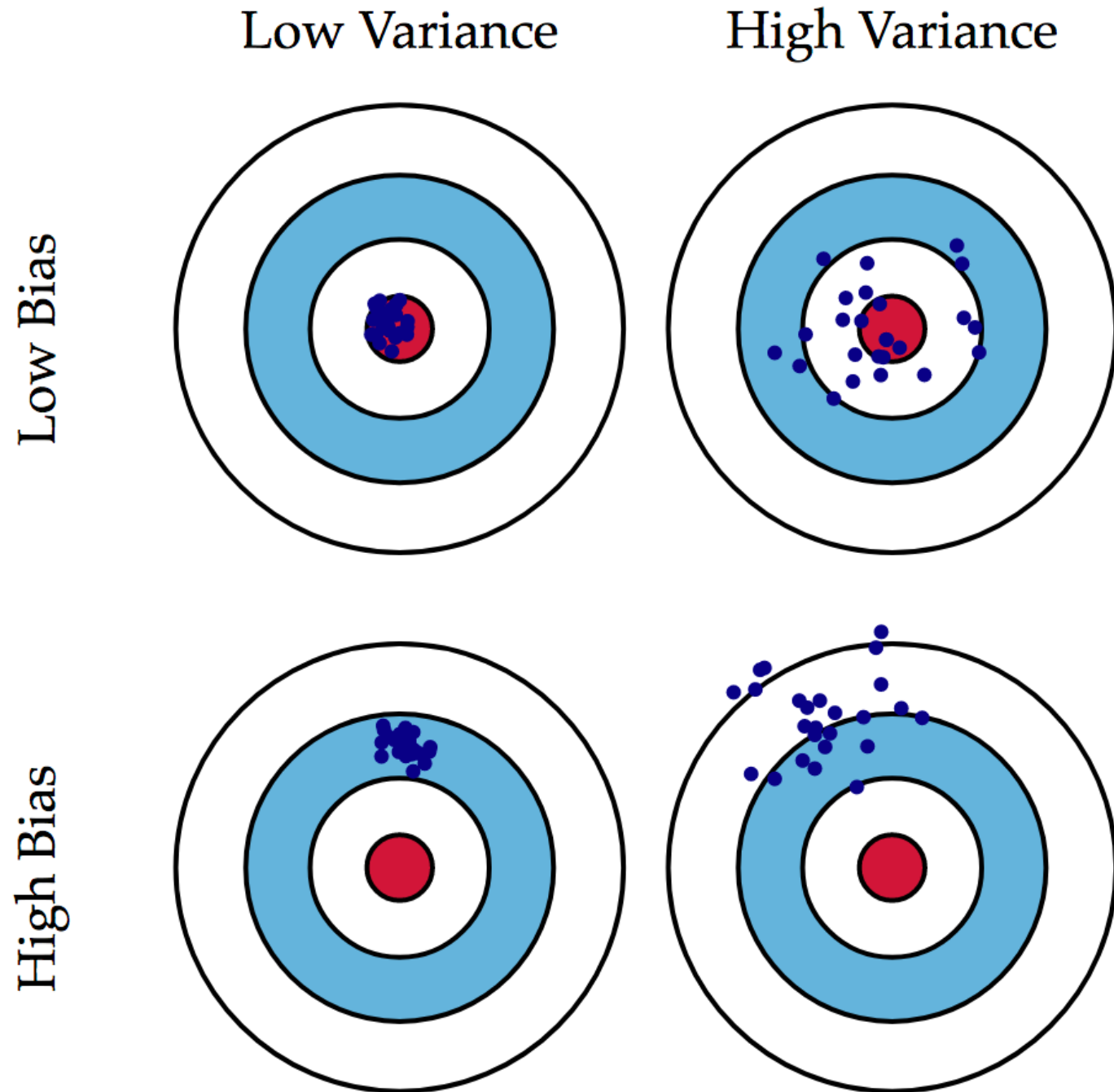
Um Fehler durch Bias bzw. durch Varianz zu verstehen, stellen wir uns vor, dass wir die **gesamte Modellierung mehrmals wiederholen**. Jedes mal ...

1. ... sammeln wir neue Daten
2. ... trainieren ein neues Modell

Wegen Zufälligkeit (Streuung) in den Datensätzen werden die Modelle i.d.R. nicht jedes Mal die gleiche Vorhersage produzieren sondern einen Bereich an Vorhersagen.

Sei x ein Datenpunkt, für welchen wir die Vorhersagen registrieren.

- **Fehler durch Bias:** Differenz zwischen der durchschnittlichen Vorhersage $h(x)$ aller Modelle h und dem wahren Zielwert.
- **Fehler durch Varianz:** Variabilität der Vorhersagen $h(x)$ im Punkt x



Bias vs. Variance

Diskussion (3–5 Min.)

Angenommen, Sie wollen vorhersagen, wie viel Prozent der potenziellen Wähler*innen für eine Randbebauung des Tempelhofer Feldes stimmen werden. Sie entscheiden sich für folgendes Vorgehen:

1. Wähle zufällig 50 Nummern aus dem Telefonbuch
2. Rufe jede Nummer an und Frage, wie die Person beabsichtigt abzustimmen

Sie erhalten folgendes Ergebnis:

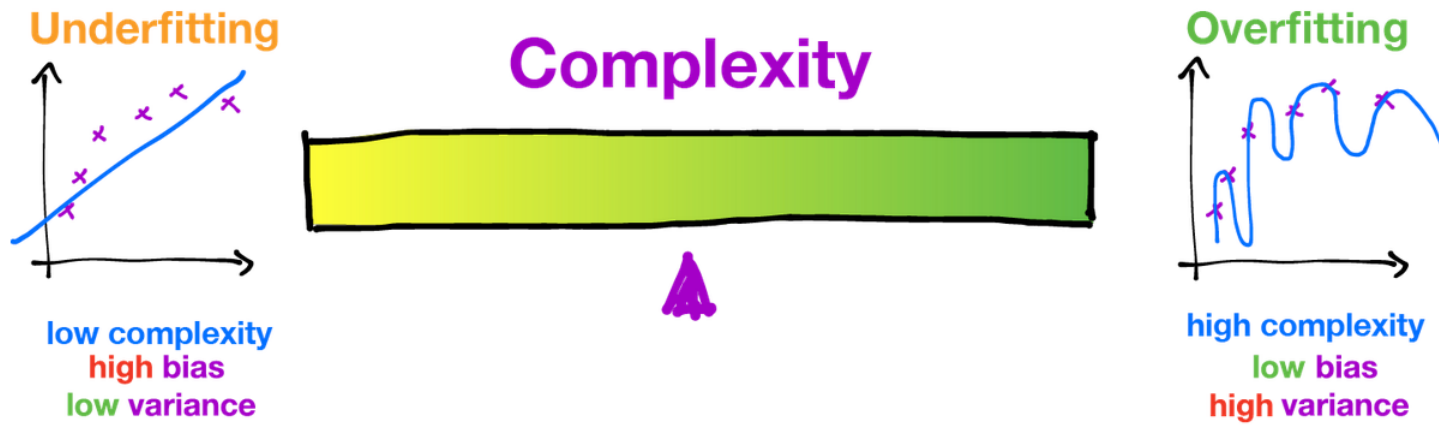
Dafür	Dagegen	Keine Antwort	Gesamt
13	16	21	50

Die geschätzte Wahrscheinlichkeit, dass für die Bebauung gestimmt wird, liegt bei ca. 0,45.

- Welche Fehlerquellen sehen Sie bei diesem Vorgehen?
- Können Sie bestimmen, ob die Fehlerquelle eher den Bias oder die Varianz erhöht?

► Einige Fehlerquellen

- Auswahl aus Telefonbuch: bezieht nur Personen mit ein, deren Nummer gelistet ist (Bias)
- Die nicht-Antwortenden auszulassen führt zu einer konsistenten Verzerrtheit der Daten (Bias)
- Wenige Daten (nur 50) führt zu hoher Varianz



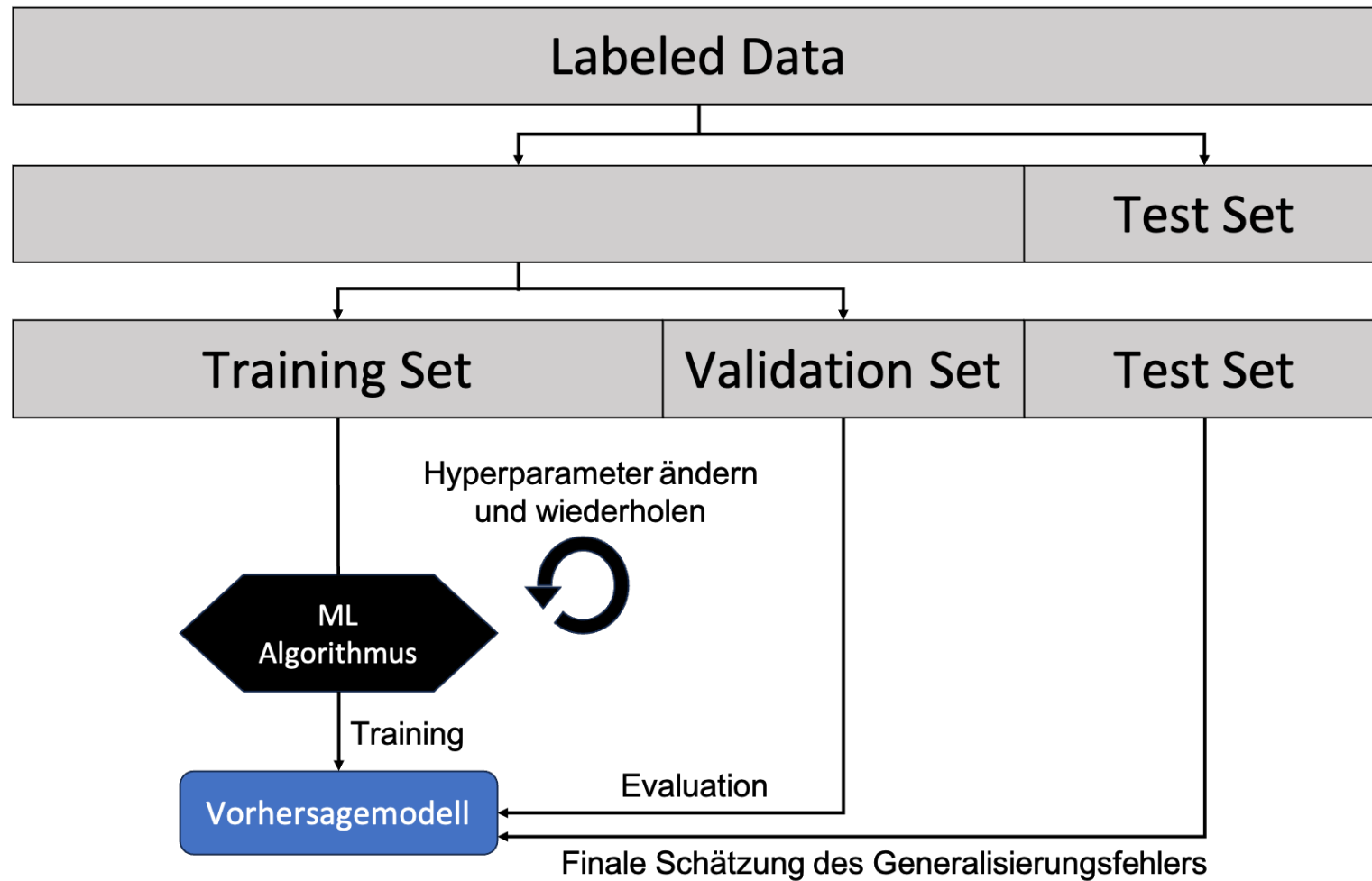
© Machine Learning @ Berkeley

Zusammenhang zwischen Over-/Underfitting und Bias/Variance

Modellauswahl

Wie entscheiden wir uns nun für das beste k bei k -NN - oder allgemeiner für die beste Konfiguration von Hyperparametern?

- Für die **Modellauswahl** (innerhalb einer oder zwischen verschiedenen Modell-Familien, oder auch Entscheidung über Vorverarbeitungsschritte) verwenden wir eine zusätzliche Menge der Daten, den sog. **validation set** oder **Validierungsmenge**
- D.h., die **Menge der gelabelten Daten** D wird aufgeteilt in:
- D_{train} : Training eines Modells (pro Modell-Familie und Hyperparameter-Auswahl)
- D_{val} : Vergleich der trainierten Modelle
- D_{test} : Schätzung des Generalisierungsfehlers



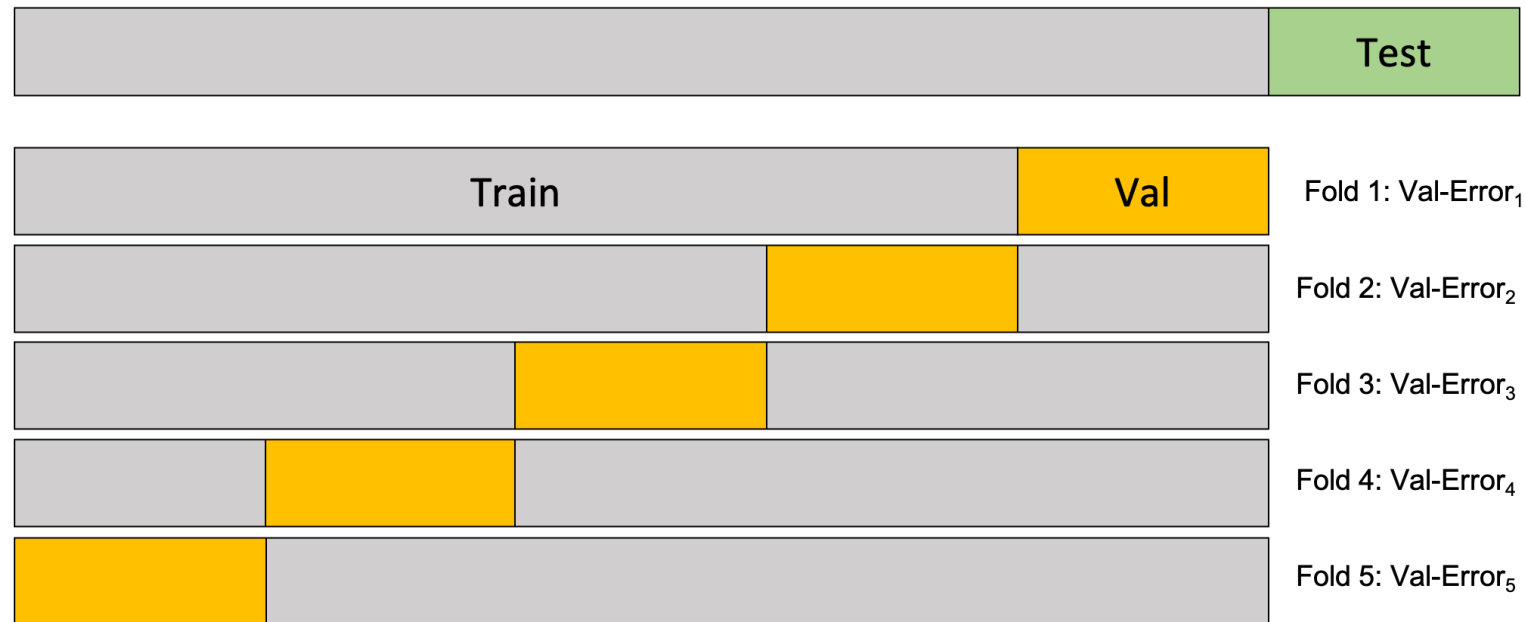
Train-Validation-Test Split. Nach abgeschlossener Modellauswahl wird das finale Modell auf der Vereinigung der Trainings- und Validierungsdaten trainiert. In Anlehnung an: Sebastian Raska and Vahid Mirjalili, Python Machine Learning, 2nd Ed. (2017), <https://github.com/rasbt/python-machine-learning-book-2nd-edition>

- Dabei gilt: je größer D_{train} , desto kleiner wird der Fehler auf den Trainingsdaten; je größer D_{val} , desto zuverlässiger ist die Schätzung des Fehlers basierend auf den Validierungsdaten.
- Daumenregel für die Aufteilung der Daten:

1. Splitte D , so dass D_{test} 20% der Daten umfasst
2. Splitte den Rest, so dass D_{train} 80% und D_{val} 20% umfassen

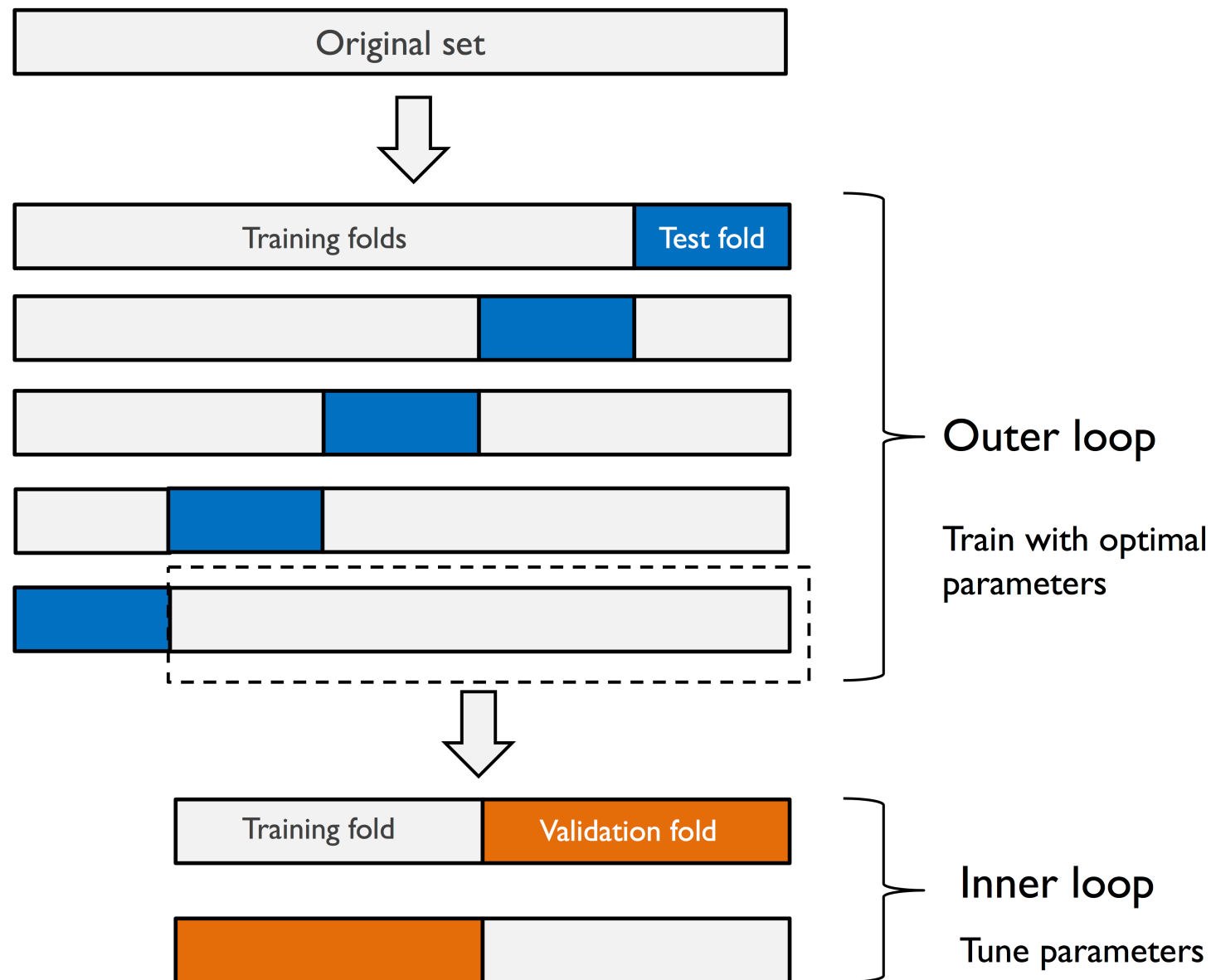
n -fold Cross-Validation

- Ist die Menge D der gelabelten Daten gering, so ist ein einfacher Split in D_{train} und D_{val} nicht zuverlässig genug. In diesem Fall wird eine sog. **Kreuz-Validierung** durchgeführt
- Dabei teilen wir die Menge $D \setminus D_{test}$ **mehrfach** in D_{train} und D_{val} auf
- Auf jedem der sog. **Training Folds** trainieren wir Modelle für die verschiedenen Werte von k (oder anderer Hyperparameter) und berechnen die Loss auf dem jeweiligen **Validation Fold**
- Diejenige Wahl von Hyperparameter, die den geringsten durchschnittlichen Val-Fehler hat, ist optimal
- Beispiel (siehe Abbildung unten): Wähle diejenigen Hyperparameter, die den mittleren Validation-Error minimieren:
$$h^* = \arg \min_h \frac{1}{5} \sum_{i=1}^5 \text{Val-Error}_i(h)$$
- Vorteil: alle Datenpunkte (außerhalb der testmenge) werden für training und für Validierung genutzt, was die Zuverlässigkeit der Modellauswahl erhöht
- Nachteil: Höherer Trainingsaufwand

Skizze der n -fold cross-validation für $n = 5$

Nested Cross-Validation

- Bei großen Datenmengen reicht ein einfacher Train-Val-Test Split
- Bei Verfahren mit sehr langen Trainingszeiten ist oftmals nicht mehr möglich
- Wenn man aber wenig Daten hat und schnell zu trainierende Modelle, so macht es Sinn, die Testdaten ebenfalls zu variieren, um den Generalisierungsfehler zuverlässiger schätzen zu können
- Dafür gibt es die sog. nested n -fold cross validation



Nested Cross Validation mit einem 5-fold split in der outer Loop und einem 2-fold Split in der inneren Loop.

Quelle: Sebastian Raska and Vahid Mirjalili, Python Machine Learning, 2nd Ed. (2017), <https://github.com/rasbt/python-machine-learning-book-2nd-edition>

Aufgaben

Aufgabe 1 - Konzepte unterscheiden

Erklären Sie den Unterschied zwischen Trainingsdaten, Validierungsdaten und Testdaten, und deren jeweilige Rolle im Modellierungsprozess.

Aufgabe 2 - Modellauswahl verstehen

Sie testen verschiedene Werte von k für k -NN und erhalten folgende mittlere Fehlerwerte:

k	Trainingsfehler	Validierungsfehler
1	0.00	0.35
3	0.12	0.28
5	0.18	0.24
7	0.21	0.25
9	0.25	0.27

Fragen:

- Welchen Wert von k würden Sie für das endgültige Modell wählen?
- Warum steigt der Trainingsfehler mit zunehmendem k ?
- Warum zeigt der Validierungsfehler ein Minimum bei $k = 5$ und steigt danach wieder?
- Was würde passieren, wenn Sie sehr große Werte für k (z. B. $k = 50$) testen würden?

Aufgabe 3 - Bewertungs des Verfahrens

In einer Publikation steht: "Wir haben unsere Hyperparameter mit 5-fach Cross-Validation optimiert und anschließend den mittleren CV-Fehler als finale Performance berichtet." Ist das korrekt? Wenn nicht, warum nicht? Wie müsste man es richtig machen?

Aufgabe 4 - Berechnung und Darstellung des Trainings- und Validierungsfehlers

Führen Sie für eine andere Feature-Auswahl des Palmer-Pinguins Datensatzes einen Train-Val-Split mit 20 % Validierungsdaten und `random_state=1` durch. Trainieren Sie für k von 1 bis 30 ein k -NN Modell auf den Trainingsdaten und berechnen Sie die Ungenauigkeit jeweils auf beiden Mengen. Stellen Sie die Ergebnisse als Liniendiagramme in einer Abbildung dar. Sie können den Code aus der Vorlesung als Vorlage verwenden.

Aufgabe 5 - Cross-Validation

Führen Sie eine 5-fold Cross Validation durch statt eines einfachen Train-Val-Splits. Geben Sie die Genauigkeit auf allen Folds aus. Welcher Wert von k ist optimal?

- [1] k -NN ist kein parametrisches Verfahren aber die Trainingsbeispiele haben eine ähnliche Funktion wie die Gewichte in einem polynomiellen Modell.